研发-动态绘制禁行区

概述

由于使用 Cartographer 进行 SLAM 建图，地图是动态生成的（pbstream 格式），无法直接修改地图文件。本方案通过订阅 Cartographer 发布的地图，在其基础上叠加 keepout mask，实现禁行区域的动态配置。

工作原理

1. **keepout_mask_publisher** 节点订阅 Cartographer 发布的 `/map` topic
2. 从 YAML 配置文件加载禁行区域的**世界坐标**
3. 将世界坐标转换为地图像素坐标
4. 生成 keepout mask 并发布到 `/keepout_filter_mask` topic
5. Nav2 的 `keepout_filter` 读取 mask 并应用到 costmap

配置步骤

1. 编辑禁行区域配置文件

编辑 `robot_nav/config/keepout_mask.yaml` 文件，定义禁行区域：

```
代码块
1   keepout_zones:
```

矩形禁行区域（推荐用于简单区域）

- name: "restricted_area_1"

type: "rectangle"

corners:

- {x: -2.0, y: -1.0} *# 左下角（世界坐标，单位：米）*
- {x: -1.0, y: 0.5} *# 右上角*

多边形禁行区域（适合不规则区域）

- name: "restricted_area_2"

type: "polygon"

vertices:

- {x: 1.0, y: 1.0}
- {x: 2.0, y: 1.5}
- {x: 2.5, y: 0.5}
- {x: 1.5, y: 0.0}
- {x: 0.5, y: 0.5}

代码块

```
**坐标获取方法：**

- 使用 `rviz2` 查看地图，鼠标悬停位置会显示世界坐标
- 使用 `ros2 topic echo /map` 查看地图元数据（origin、resolution）
- 在 rviz2 中添加 `Map` 显示，使用 `2D Nav Goal` 工具查看坐标

### 2. 构建节点

```bash
cd ~/agrobot_ws
colcon build --packages-select robot_nav
source install/setup.bash
```

注意： 如果编译时提示找不到 `yaml-cpp`，需要安装：

代码块

```
sudo apt-get install libyaml-cpp-dev
```
```

3. 启动导航（已包含 keepout 支持）

```
代码块
1   ros2 launch robot_nav nav03_navigation.launch.py
```

节点会自动启动：

- `costmap_filter_info_server` - 提供过滤器信息
- `keepout_mask_publisher` - 发布 keepout mask

验证配置

检查节点是否启动

```
代码块
1   ros2 node list | grep -E "keepout|costmap_filter"
```

应该看到：

- `keepout_mask_publisher`
- `costmap_filter_info_server`

检查 Topics

```
代码块
1
```

检查 mask 是否发布

ros2 topic echo /keepout_filter_mask --once

检查 filter_info

ros2 topic echo /costmap_filter_info --once

代码块

```
### 在 RViz2 中可视化

1. 启动 RViz2: `rviz2`
2. 添加显示:
   - `Global Costmap` → Topic: `/global_costmap/costmap`
   - `Local Costmap` → Topic: `/local_costmap/costmap`
   - `Map` → Topic: `/keepout_filter_mask` （应该看到禁行区域为黑色）

3. 尝试导航到禁行区域附近，路径规划应该会避开这些区域

## 动态更新禁行区域

### 方法 1: 修改配置文件并重启节点

1. 编辑 `keepout_mask.yaml`
2. 重启 `keepout_mask_publisher` 节点:
   ```bash
```

ros2 service call /keepout_mask_publisher/_lifecycle/change_state lifecycle_msgs/srv/ChangeState "{transition: {id: 1}}" *# 关闭*

ros2 service call /keepout_mask_publisher/_lifecycle/change_state lifecycle_msgs/srv/ChangeState "{transition: {id: 3}}" *# 重启*

代码块

```
### 方法 2: 使用参数重新加载（需要节点支持）

当前版本需要修改配置文件后重启节点。未来可以扩展为支持服务接口动态添加/删除禁行区域。

## 故障排查

### Mask 未生效

1. **检查节点是否运行**:
```
```

```bash
ros2 node list
```

6. **检查配置是否加载：**

查看节点日志：

```
ros2 topic echo /rosout | grep keepout
```

7. **检查坐标是否正确：**
   - 确保坐标使用地图坐标系（map frame）
   - 确保坐标范围在地图范围内
8. **检查 mask topic：**

```
ros2 topic hz /keepout_filter_mask
```

禁行区域位置不对

- 检查坐标系统：确保使用 map frame 的世界坐标
- 检查地图原点：查看 `/map` topic 的 `info.origin` 字段
- 使用 rviz2 工具验证坐标

编译错误

如果提示找不到 `yaml-cpp`：

```
sudo apt-get update
sudo apt-get install libyaml-cpp-dev
```

高级用法

禁用 Keepout Filter

在启动文件中设置参数：

```
代码块
1   'enable_keepout': False,
```

或在 launch 时传递参数：

```
代码块
1   ros2 launch robot_nav nav03_navigation.launch.py enable_keepout:=false
```

自定义 Map Topic

如果 Cartographer 发布的地图 topic 不同，可以修改：

```
代码块
1   'map_topic': 'your_map_topic',
```

示例场景

场景 1：保护工作区域

在工作台周围设置矩形禁行区域：

```
代码块
```

```yaml
  - name: "workbench_area"
    type: "rectangle"
    corners:
      - {x: 2.0, y: 1.0}
      - {x: 4.0, y: 3.0}
```

场景 2：避开不规则障碍

使用多边形定义不规则区域：

代码块

```yaml
  - name: "irregular_zone"
    type: "polygon"
    vertices:
      - {x: -1.0, y: -1.0}
      - {x: 0.0, y: -0.5}
      - {x: 0.5, y: 0.0}
      - {x: -0.5, y: 0.5}
```

未来扩展

可以考虑添加：

- ROS2 服务接口动态添加/删除禁行区域
- 可视化工具在地图上交互式绘制禁行区域
- 支持圆形、椭圆形等更多几何形状
- 支持临时禁行区域（带时间限制）